

Head-to-head Evaluation of Translation-First vs Agentic Generate→Validate→Repair Pipelines for Intent-Driven NetOps Changes — Validator-Acceptance Parity under Shared-Candidate Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-archived paired experiment (CAND-2026-001-prereg-v1) that compared two operational architectures for intent→configuration NetOps changes across heterogeneous vendor CLIs and Mininet-derived topologies: (A) translation-first (constrained vendor DSL → formal verifier → solver) and (B) agentic generate→validate→repair (hosted LLM + deterministic validators). The run declared gpt-5-mini for generation and deterministic_netops_validator_v5 for deterministic end-to-end correctness checks; preregistration used shared_initial_candidate semantics so each task pair received a single LLM candidate shared between arms. Execution completed 240 episodes (120 paired tasks) with model_calls_used = 120 (one shared generation per pair). Deterministic scoring produced contingency counts n11 = 120, n10 = 0, n01 = 0, n00 = 0 (baseline = 120/120; guarded = 120/120). Paired difference (A - B) = 0.00; paired-bootstrap 95% CI = [0.00, 0.00] (10,000 resamples, seed 1729); exact McNemar two-sided p = 1.0. Because generation was intentionally shared, these outcomes measure validator-acceptance parity for identical candidates rather than independent generator or repair robustness. We provide artifact-grounded evidence (execution and analysis manifests, per-task validator traces), an explicit evidence-to-claim mapping table, a nearest-work comparison matrix, and preregistered protocol modifications required to obtain a discriminating head-to-head comparison in follow-ups.

I. INTRODUCTION

Automating intent→configuration translation and safe sequencing of multi-vendor NetOps changes is operationally critical: production networks require both correctness guarantees (reachability, isolation, absence of transient safety violations) and flexibility to express diverse intents across vendor CLIs. Two architectural approaches are common: translation-first pipelines that restrict generation to a vendor DSL and apply formal verification/solvers, and agentic generate→validate→repair pipelines that centralize generation in an LLM and rely on deterministic validators plus bounded repair. Prior literature emphasizes both deterministic guarantees (formal verification) and the promise of LLM-driven flexibility, but direct, preregistered comparisons that produce artifactized per-task validator traces are rare.

This study’s preregistered head-to-head question was: for intent-driven network changes across heterogeneous vendor

CLIs and realistic emulator topologies, which architecture yields higher end-to-end operational correctness (measured by deterministic validation: reachability and policy isolation) — (A) translation-first or (B) agentic generate→validate→repair? The preregistration (CAND-2026-001-prereg-v1) fixed a paired design (N = 120 paired tasks), the primary estimand (paired difference in binary end-to-end correctness A - B), the generator (gpt-5-mini), the deterministic validator (deterministic_netops_validator_v5), and the analysis executor (paired_binary_analysis_v1).

A decisive preregistered design choice was shared_initial_candidate semantics: each paired task used a single LLM candidate generated once and supplied identically to both architectures. This choice deliberately removes generator variance from the confirmatory estimand, turning the experiment into a validator-acceptance parity test for identical candidates rather than an independent generator or repair efficacy comparison. The remainder of the paper (Methods, Results, Discussion) makes that restriction explicit and maps preregistered hypotheses to the measured estimand with artifact references.

II. RELATED WORK

Two research traditions frame our contribution and motivate the experimental axes used in this study: (1) formal translation and verification systems that prioritize deterministic guarantees, and (2) LLM-driven agentic NetOps work that prioritizes flexible generation and iterative repair. Below we expand these threads and contrast their experimental semantics, instrumentation, and typical evaluative practices—focusing specifically on the axes used by our preregistered design (independent generation sampling, repair-loop instrumentation, validator acceptance focus, and emulator/hardware realism).

Formal translation→verification. A long line of work treats network configuration synthesis and verification as a constrained-synthesis problem: generate configurations in a restricted DSL, then apply solver-based or symbolic verification to obtain soundness guarantees for reachability and isolation. Seminal systems such as SymNet and follow-on "Don’t

Mind the Gap" emphasize determinism and programmatic representations of forwarding behavior; they enable localized, incremental checks and provide formal accounts of when synthesized configuration changes preserve invariants [10,11]. More recent programming-language and verification advances (e.g., Weighted NetKAT and related quantitative verification work) extend the expressiveness of DSLs and enable richer objectives (cost, weighted properties) while retaining verifier semantics that produce binary accept/reject decisions or counterexamples suitable for solver feedback [5]. The experimental semantics common to these systems are: (i) a constrained generation surface (DSL) that bounds output space; (ii) formal validators that operate on an abstract semantics of device behavior; and (iii) low generator variance because synthesis operates inside the DSL or under solver guidance. Empirical comparisons in this family typically focus on solver scalability, counterexample diagnostics, and incremental verification throughput rather than stochastic generator variance.

LLM-driven and agentic NetOps. In contrast, the recent wave of LLM-powered NetOps and AIOps research treats translation as a flexible natural-language→CLI mapping problem and emphasizes sample-based generation plus iterative repair. Representative recent works explicitly instrument generate→validate→repair pipelines and multi-agent verifier loops to reduce hallucination and unsafe proposals [3,4]. These LLM-centric evaluations commonly exercise independent generation sampling (multiple model calls per condition), record repair iteration traces, and measure repair coverage and transient unsafe-proposal rates as operational metrics. Because LLM outputs are stochastic and can manifest structural errors, LLM-centric experiments often allocate larger model-call budgets, instrument per-generation diagnostics (tokens, parse errors), and adopt deterministic validators as downstream safety gates. Critically, LLM experiments that compare pipelines frequently differ in whether they (a) compare independent generator draws per arm (which measures generator+orchestrator combined performance) or (b) hold the generator output fixed to contrast downstream orchestration and verification behaviors.

Experimental axes and their methodological consequences. Prior work therefore occupies distinct points along the four experimental axes we emphasize: independent generation sampling, repair-loop instrumentation, validator-acceptance focus, and emulator/hardware realism. Translation-first systems score highly on deterministic guarantees and low generator variance (because synthesis is constrained), while LLM-agent experiments emphasize generator sampling, repair coverage, and stochastic robustness. Mahmood & Song (IFIP Networking 2026) report a self-correcting multi-agent verifier pipeline instrumenting repair loops and acceptance thresholds; INTA (ICNP 2025) and related intent→configuration studies evaluate translation strategies and LLM agents with independent generation sampling to capture generator variance [3,4]. Weighted NetKAT and related formal frameworks supply the deterministic foundation and are commonly used as the downstream verifier in translation-first evaluations [5]. These verified works

are the direct methodological relatives of the present run.

Positioning of the present run. The distinguishing experimental decision in our preregistration is shared_initial_candidate semantics: for each paired task we generated a single LLM candidate and supplied that identical candidate to both the translation-first and agentic arms. Methodologically, this places our run orthogonally to independent-generation LLM experiments: rather than estimate generator variance or repair coverage, the run isolates differences in downstream acceptance and orchestration given identical candidate proposals. This choice mirrors verifier-parity regression checks frequently used when deploying new validators or orchestration stacks (i.e., when operators need to know whether swapping validators/orchestrators changes acceptance outcomes for the same proposals). Nearest works that instrument repair loops and sample independently remain complementary: to discriminate generator robustness or repair efficacy one must change the generation semantics (independent per-arm draws and nonzero repair budgets) and correspondingly increase model_call budgets and repair instrumentation. Our discussion and preregistered recommended protocol modifications make these resource and instrumentation differences explicit.

Artifact and evaluation practices across prior work. A practical contrast concerns archival and per-task artifactization. Formal translation studies typically archive DSL programs, constraints, and counterexamples produced by solvers; LLM agent studies archive multiple model responses, repair prompts/traces, and validator logs. The present run follows the LLM-artifact practice but intentionally reduces generator artifact multiplicity by using one shared response per pair; archival traces therefore emphasize per-task validator snapshots (validation_before/after), repair_applied flags, and the deterministic acceptance decision. This archival focus enables reproducible validator-parity claims while preserving the ability to modify generation semantics in follow-ups to test the broader preregistered hypotheses about independent generator and repair differences.

III. METHODOLOGY

Preregistration and execution contract. The preregistration (CAND-2026-001-prereg-v1) fixed the primary estimand (paired difference in binary end-to-end correctness $A - B$), $N_{\text{confirmatory}} = 120$ paired tasks, generator = gpt-5-mini, validator = deterministic_netops_validator_v5 (v5.0), adapter_family = hosted_netops_gvr_v1, maximum_model_calls = 240, and shared_initial_candidate semantics. The preregistration also specified paired_binary_analysis_v1 (bootstrap percentile CIs and exact McNemar test) and a power plan targeting a ~0.15 paired difference with $N=120$. The preregistration artifact is archived and its checksum is recorded in the execution manifest.

Task generation and difficulty calibration. Confirmatory tasks (120 pairs) were generated by deterministic_netops_task_generator_v5. Each manifest entry (execu-

tion/task_manifest.jsonl) contains: intent text, canonical reference answer, initial and expected final states, required safe sequences, and difficulty metadata (changed_interface_count, dependency_rule_count, level: medium/hard/very_hard). Representative entries (e.g., task-000001) and their canonical reference answers are archived and used by the deterministic validator as ground truth for normalized comparison.

Generation and shared-candidate semantics. The orchestration implemented shared_initial_candidate: for each pair the system made a single LLM call and distributed the identical textual candidate to both downstream arms. Execution accounting (execution/execution_manifest.json) records planned_episode_count = 240, completed_episode_count = 240, and model_calls_used = 120 (one shared generation per pair). Provider audit (analysis/deterministic_reconciliation.json) reports hosted_model_call_event_count = 120 and cross_condition_cache_key_reuse_observed = false, consistent with execution isolation.

Validator rules and scoring. The deterministic_netops_validator_v5 applies full_intent_constraint_validation: syntactic normalization, required command inclusion, ordering/dependency checks (make-before-break, shutdown→modify→restore), final state consistency, and emits validation_after.valid (boolean). A configuration is scored success (1) only when validation_after.valid == true. Per-task scoring JSONs (execution/scoring/task-000XXX-*.json) record normalized_response, parsed_command_count, required_command_count, validation_before/after snapshots, repair_applied flags and the scorer id/version. These files are the canonical evidence for every per-task decision and are archived in the execution bundle.

Analysis plan and exact procedures. Analysis followed paired_binary_analysis_v1, computing contingency counts (n11, n10, n01, n00), paired difference $A - B$, bootstrap percentile 95% CI (10,000 resamples, bootstrap_seed = 1729), and exact McNemar two-sided test. Analysis artifacts include analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl (bootstrap parameters), and analysis/deterministic_reconciliation.json. Secondary estimands (repair coverage, mean repair iterations) were preregistered; their measurement depends on nonzero repair_applied flags in scoring JSONs.

IV. RESULTS

Execution accounting and primary numeric outcomes. The execution manifest (execution/execution_manifest.json) reports: planned_episode_count = 240, completed_episode_count = 240, failed_episode_count = 0, model_calls_used = 120, and artifact_hash_summary. Deterministic reconciliation (analysis/deterministic_reconciliation.json) confirms complete_pair_count = 120 and provider audit counts consistent with the single shared generation stage.

Deterministic scoring and contingency. The primary contingency (analysis/paired_contingency_table.csv) is: • n11 = 120 (accepted by both arms) • n10 = 0 (accepted only by guarded) • n01 = 0 (accepted only by baseline) • n00 = 0 (rejected by both) Marginal success rates are baseline = 120/120 = 1.0 and guarded = 120/120 = 1.0. Paired difference ($A - B$) = 0.00. The bootstrap percentile 95% CI (10,000 resamples, seed 1729) reported in analysis/analysis_log.jsonl is [0.00, 0.00], and the exact McNemar two-sided $p = 1.0$.

Repair and secondary outcomes. Aggregated repair_applied count = 0 for all 240 episodes; every execution/scoring/*.json entry shows repair_applied = false. Therefore preregistered secondary estimands about repair coverage (H2) are unobserved in this run. Representative per-task traces (e.g., execution/scoring/task-000001-baseline.json) show shared_initial_candidate equal to the normalized reference modulo ordering and validation_after.valid = true.

Contamination and missingness diagnostics. The preregistered contamination checks produced no flags: analysis/contamination_summary.csv is empty and analysis/missingness_summary.csv records complete_pairs = 120 with no observed failures. Provider metadata reports cache_miss_count = 120 and cross_condition_cache_key_reuse_observed = false, supporting execution isolation and adherence to the contamination_plan.

Representative example. Pair-000001 artifacts (execution/responses/task-000001-shared-initial.txt and execution/scoring/task-000001-baseline.json) document the shared candidate, normalized_response, parsed_command_count = 4, required_command_count = 4, and validation_after.valid = true. Similar representative artifacts for additional pairs (task-000002, task-000003, etc.) are archived and referenced in the execution manifest.

V. DISCUSSION

What the run measures — and what it does not. Because shared_initial_candidate semantics were used, the preregistered confirmatory estimand reduces to validator-acceptance parity: given identical LLM candidates, do downstream orchestration and verification paths differ in deterministic validator acceptance? The observed complete concordance (n11 = 120) answers this restricted question: both validators accepted the identical candidates for every paired task. The run does not exercise independent generator variance, sampling effects, or repair efficacy; therefore claims about which architecture would produce more correct initial candidates or achieve higher repair coverage when generating independently are untested.

Evidence-to-claim mapping (compact table). Table 2 (in manuscript) maps preregistered hypotheses/estimands to measured observables and archived artifacts. Example mappings: H1 (paired difference $A - B$) → measured estimand = validator-acceptance parity under shared_initial_candidate; supporting artifacts:

analysis/paired_contingency_table.csv and analysis/condition_summary.csv (n11=120,n10=0,n01=0,n00=0); verdict: tested but restricted. H2 (agentic repair coverage advantage) → observed repair_applied count = 0 in execution/scoring/*.json; verdict: unobserved in this execution.

Operational implications and protocol guidance for discriminating comparisons. The artifact bundle documents a reproducible protocol for validator regression and orchestrator equivalence checks under fixed candidates. To test independent-generation and repair differences the pre-registered protocol must be modified (and these modifications can be preregistered and audited): (i) remove shared_initial_candidate so each arm receives an independent LLM generation per pair (model_calls_used would rise from 120 to 240 for initial candidates); (ii) allow multiple sampled initial generations per arm to estimate generator variance; (iii) enable and instrument repair iterations so repair_applied counters become informative; (iv) increase emulator fidelity or include hardware runs to reduce simulator-to-hardware contamination risk. Each modification yields verifiable changes in execution artifacts (model_calls_used, hosted_model_call_event_count, nonzero repair_applied flags) and therefore supports reproducible discrimination across architectures.

Threats to validity (concise). Internal validity: shared_candidate removes generator variance by design; this is a feature not a flaw but narrows the estimand. Construct validity: deterministic validator acceptance operationalizes a conservative correctness definition but omits human-facing properties (explainability, maintainability). External validity: synthetic tasks and Mininet-derived emulation plus a single model (gpt-5-mini) and single validator (deterministic_netops_validator_v5) limit generality. Conclusion validity: degenerate concordance (all pairs concordant) produces point mass CI and $p = 1.0$; these statistics correctly describe the observed data but do not imply production-level equivalence under independent operation.

Reproducibility and audit artifacts. All primary artifacts are archived and referenced by execution/execution_manifest.json: preregistration (CAND-2026-001-prereg-v1), execution/model_configuration.json (declared_model_version = gpt-5-mini), execution/task_manifest.jsonl, execution/responses/*, execution/scoring/*.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl, and analysis/deterministic_reconciliation.json. Per-task validator traces (execution/scoring/task-000XXX-*.json) are the canonical records for each scored decision. The analysis executor parameters (bootstrap_resamples = 10000, bootstrap_seed = 1729) are recorded in analysis/analysis_log.jsonl and enable deterministic recomputation of the reported CI.

VI. LIMITATIONS

- Preregistered shared_initial_candidate semantics were used; this intentionally removes generator variance and prevents this run from assessing independent generation robustness differences between architectures.
- The task corpus was synthetic and executed in an emulator (Mininet-derived topologies and public vendor example grammars); emulator-to-hardware divergence limits external validity to production devices.
- A single LLM (gpt-5-mini) and a single deterministic validator (deterministic_netops_validator_v5) were used; results do not imply multi-model or multi-validator generality.
- No repairs were applied in this run (repair_applied = false in per-task traces); preregistered secondary estimands about repair coverage remain unobserved for this execution.
- The binary deterministic validator acceptance metric does not capture operator-facing properties such as explanation quality, maintainability, or human trust, which matter in production but were outside the metric used here.

VII. CONCLUSION

We executed a preregistered, artifactized paired experiment comparing translation-first and agentic generate→validate→repair NetOps pipelines under shared_initial_candidate semantics. For 120 paired tasks deterministic validators accepted identical LLM candidates in both arms for every pair ($n_{11} = 120, n_{10} = n_{01} = n_{00} = 0$), yielding paired difference 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar $p = 1.0$). No repairs were applied (repair_applied = false for all episodes); preregistered hypotheses about repair coverage are therefore unobserved. The run precisely measures validator-acceptance parity under controlled shared candidates and provides a reproducible artifact bundle and explicit protocol modifications for future discriminating comparisons that exercise independent generation and repair coverage.

DISCLOSURE STATEMENT

This research was executed autonomously after the autonomy lock under the frozen master prompt archived with the run provenance. Preregistration: CAND-2026-001-prereg-v1 (archived and used to freeze the design; preregistration_sha256 recorded in the execution manifest). Generation used gpt-5-mini (declared_model_version in execution/model_configuration.json); provider record 'openai_responses'. The hosted adapter family was hosted_netops_gvr_v1. Deterministic artifacts included deterministic_netops_task_generator_v5 and deterministic_netops_validator_v5 (validator_version 5.0). The run deliberately used the preregistered shared_initial_candidate generation semantics: both pipeline arms received the same initial LLM candidate per task; execution accounting shows model_calls_used = 120 (one shared generation per pair) while planned episodes were

240. Primary analysis used paired bootstrap percentile CIs (10,000 resamples, seed 1729) and an exact McNemar two-sided test executed by `paired_binary_analysis_v1`. No human manually edited, rewrote, or post-processed technical content after the autonomy lock. The Research Director selected the broad topic family and constrained the initial master prompt prior to the autonomy lock; the autonomous run performed literature review, final research-question selection, preregistration, code generation, experiment execution, analysis, peer review, and manuscript drafting after the lock. Immutable reference to the initial master prompt: `provenance/master_prompt.txt` (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). The full execution and analysis artifact bundle is archived and referenced by the execution manifest included with this submission.

REFERENCES

- [1] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, David Walker, "Don't Mind the Gap", 2016, 10.1145/2934872.2934909
- [2] Radu Stoenescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881
- [3] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391
- [4] Umar Mahmood, Wang-Cheol Song, "A Self-Correcting Multi-Agent LLM Pipeline for Verified Kubernetes Network Policy", 2026, 10.23919/ifipnetworking70592.2026.11578993
- [5] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318